

COM668 Computing Project

Assessment Task 2 – Challenge Definition Report

Project title:	HomeRights-Ai
Student name:	Abdullah Al Sayeed
Student ID number:	B00956522
PSG identifier:	PSG-Q25
Mentor name:	Hemalatha Raj
Course title:	Computing Project
Year of submission:	2025/26
Word Count :	3694

Declaration:

I declare that this is all my own work. Any material I have referred to has been accurately referenced and any contribution of Artificial Intelligence technology has been fully acknowledged. I understand the importance of academic integrity and have read and understood the University's General Regulation: Student Academic Integrity and the Academic Misconduct Procedure. I understand that I must not upload my work before, during or after submission to any unapproved plagiarism detectors or answer sharing platforms, or equivalent, and that only University-approved platforms should be used.

Contents

Contents.....	<i>i</i>
1. Introduction	1
1.1. Aim	1
1.2. Problem description.....	1
1.3. Scope	2
1.4. Intended outcome.....	2
1.5. Objectives	3
2. Contextual research	3
2.1. Context investigation.....	3
2.2. Supporting theory research.....	4
2.3. Similar products review	4
3. Software definition	4
3.1. Requirements analysis	4
3.2. Risk analysis	5
3.3. Design	7
3.4.	7
3.5.	8
3.6.	9
4. Planning.....	9
4.1. Methodology	9
4.2. Initial plans	10
References	13
Appendices.....	14

1. Introduction

1.1. Aim

The objective of the project is to design and develop a full-stack web site application named HomeRights Ai, which is used to make accessing the UK housing law easier to tenants. The site will be used as a form of interactive information management system which users can easily search, read and comprehend their legal rights in plain English. One of the other objectives of the project is to establish a content management environment where the legal contributors and administrators can update and maintain the right information with ease. Through hosted open-source programming (React, Node.js, Python and MongoDB) built on Microsoft Azure, the project aims at creating a scalable, accessible, and educational digital system that will enable inhabitants of the UK to make informed housing choices.

The idea behind the HomeRights AI project is to address a massive obstacle that most tenants in the UK continue to face problem over, namely the inability to get their heads round the housing legislation when it is presented in high level legal terms. Creating a complete stack web application, which will provide this info in simple, easy-to-understand language, will provide renters with a handy tool that can guide them in their daily decision-making. It has integrated interactive features like topic based browsing, clear explanatory outputs, postcode based support and document generation in order that users can confidently search and use the legal knowledge. One of the most important aspects of the system is its content-management environment that enables the legal contributors and administrators to keep the guidance up to date, to add new content and to keep it accurate as the law changes, making the app up to date and credible. Technically, the project presents the development of a modern cloud-based stack with React, Node.js, Python, and MongoDB and is hosted on Microsoft Azure. This architecture provides a scaled out solution, a high availability infrastructure and the ability to add new services or APIs in the future. We can provide access to a large user base, such as individuals with different digital or literacy requirements, by focusing on accessibility, i.e. the design that is WCAG-compliant, and the principles of clear language. In general, the project is one of the examples of how open-source technologies can be used to create significant social change by bringing awareness of the law and giving people power.

1.2. Problem description

In the UK, thousands of tenants face challenges in understanding their housing rights due to the complexity of legal documents and fragmented access to reliable advice. While organisations like Shelter and Citizens Advice provide excellent guidance, their websites can be overwhelming and often lack a unified search system that connects legal explanations with local support options. This results in tenants spending valuable time navigating multiple sources for answers.

Additionally, administrative bodies and community support services struggle to maintain updated, easy-to-read resources. This makes it difficult to ensure that legal advice is both accurate and accessible. The HomeRights platform addresses this gap by creating a centralised database of simplified housing laws and related support contacts, enabling efficient information retrieval through a user-friendly interface.

The project also considers accessibility barriers such as literacy levels and technical experience. By offering plain-language summaries, search filters, and a clear visual layout, HomeRights Ai bridges

the gap between complex legal frameworks and everyday users. For administrators, it provides the necessary tools to manage data in real time without the need for advanced technical expertise.

1.3. Scope

The scope of the HomeRights Ai MVP focuses on three main user roles:

1. Guest Users who can browse, search, and view housing law summaries with interactive chat bot.
2. Registered Users who can save topics and submit feedback.
3. Administrators/Editors who can create, update, and delete law summaries and manage local support listings.

The backend will include RESTful API endpoints for CRUD operations on law topics and user data. MongoDB will store structured information in JSON format, making it easy to query and manage. The frontend, built with React, will communicate with the backend through secure API calls. Hosting will be handled by Azure App Service and Azure Database for MongoDB, ensuring reliable cloud performance.

The MVP will not initially include fully AI-driven search or automated data imports but will have basic trained Ai initially and will be designed to integrate such features slowly in future development phases.

The HomeRights AI MVP is deliberately kept small to ensure that we can get it done on time and still be able to connect with actual users. There are three types of users who can use the site which are: Guest Users, Registered Users and Administrators hence we have designed the site based on the entire flow, and made the site open yet in control. Visitors are free to delve into the housing issues and discuss the matters with the bot to get easy legal answers. So it comes in handy even without signing up. Once you register, you receive additional personalization, such as bookmarking subjects and providing comments that are used to understand what to work on and, as a result, the project will expand with the needs of people.

Admins and Editors will be in the front line of controlling the content, and they will receive the rights to create, update, or delete housing summaries and maintain the local support listings up-to-date. That makes the legal info correct in the long run. The technology architecture is a standard RESTful API, therefore the front and back end remains clean and simpler to maintain. The JSON-ish structure of MongoDB allows us to store the subjects of the law, user accounts and support services effectively. Although the MVP is just a basic AI, we are strategizing its upgrades in the future since at a later stage we can add more advanced AI search, recommendation engines, and automatic updates without breaking anything.

1.4. Intended outcome

The intended outcome is a fully functional prototype of the HomeRights platform that demonstrates how technology can simplify the management and delivery of housing law information. The platform will provide:

- A clear, searchable database of UK housing law topics.
- Role-based access control to protect data and manage content.
- Data analytics features, such as view counts, to track popular topics.
- A modern, responsive web interface that works across devices.

- Ultimately, the project aims to improve public awareness of housing rights and reduce confusion around legal processes. It also demonstrates the value of open data integration and full-stack development for solving real-world social challenges.

1.5. Objectives

To achieve this aim, the following objectives have been identified:

- Collect and structure publicly available UK housing law data and local support information.
- Design and implement a MongoDB database schema that supports CRUD operations.
- Develop a RESTful API in Node.js for managing legal content and user data.
- Create a responsive React frontend for searching, viewing, and managing law topics.
- Implement user authentication and role-based access control (RBAC) for secure access.
- Host the full-stack application on Microsoft Azure with appropriate scaling and deployment.
- Test and evaluate the system's usability, reliability, and performance as a working MVP.

2. Contextual research

2.1. Context investigation

The usual result is that tenants receive documents that are difficult to access: tenancy agreements full of legalese, or eviction notices with strict time limits and a set of regulations. Online assistance is handy, though it is spread out all over a variety of websites. As an example, Section 21 and Section 8 notices have step-by-step instructions on how to do them, which are provided by Citizens Advice, however, you must know what type of notice you have (Citizens Advice, n.d.). Shelter goes further and describes how unfair terms are realized in tenancy agreements (Shelter, 2023). The official guidance and the current policy changes are available on GOV.UK (GOV.UK, 2019). These sources are good yet individuals/person continue to waste time by flipping pages. Policy itself is shifting. Royal Assent to the Renters' Rights Act was made on (27 Oct 2025. GOV.UK) presents the key changes and indicates that some of them will come into effect later, and a timeline is published (GOV.UK, 2025). The complete legislation resides on legislation.gov.uk (Legislation.gov.uk, 2025). The update page of Shelter also states that numerous updates are to be implemented since Spring 2026, which explains the reason why you need to have specific date labels in the application so that users can understand what is up to date when they read it (Shelter, 2025). In conclusion, any online tool must show the sources and dates in a clear way and avoid the claims of the law that is not in place yet. Consumer protection is also important. A tenancy term that lops one side of the tenancy by a large margin may be an unfair term that is not enforced. The professional advice of Shelter concludes the role of unfair conditions in housing (Shelter, 2023). The application does not need to determine whether a word is illegal, it can mark the possible danger and redirect to the reliable websites to get it verified. Finally, privacy and accessibility. A simple housing tool must adhere to (WCAG 2.2) fundamentals, such as clear headings, accessibility with a keyboard, visible focus, and good colour contrast, which will ensure all people, no matter the device used, can use it (W3C, 2024; WAI, 2023). The application should also make personal data as minimal as possible according to the UK data protection law and be transparent about what it can and cannot do, such as having a legal disclaimer that it is not a legal service.

2.2. Supporting theory research

Plain language and UX. Plain English reduces the hard legal language load: short sentences, headings matching questions of users, step lists in the form of bullets, and examples. (WCAG 2.2) provides practical guidelines that we can test: label form fields, provide clear error messages, make mobile buttons large, and ensure everything is key board friendly (W3C, 2024; WAI, 2023; GOV.UK Service Manual, n.d.).

Safety and transparency. The rules-first approach makes the MVP cautious.

The Explain feature must:

- (1) provide a quote or a summary of the text that has been copied
- (2) specify things that need to be checked twice
- (3) provide a link to GOV.UK/Citizens Advice/Shelter; and
- (4) include a salient notice that this is not legal advice.

This is the best practice of developing trustful tools that are exposed to the public.

2.3. Similar products review

GOV.UK: credible and current, the users may have to specify precise words to search (GOV.UK, 2025; GOV.UK, 2019).

Citizens Advice: step-by-step instructions (e.g. Section 21 and general eviction pages) that are easy to read (Citizens Advice, n.d.).

Shelter Legal: informative legal pages (e.g., unfair terms; the overview of security of tenure; news on Act changes) (Shelter, 2023; Shelter, 2024; Shelter, 2025).

Gap: a location where a tenant is able to fasten a fragment of a notice or a contract and right away receive plain-English advice with reliable links and dates, also nearby assistance search. The application must ensure that it is not a law firm or barrister.

3. Software definition

3.1. Requirements analysis

Functional (MVP):

- Write and describe it- the system is exceedingly easy. You simply paste your text in the box and the API returns a brief overview, the problem areas of interest, and convenient links to GO-UK, Citizens Advice or Shelter. The response goes even as far as to provide date-stamped links (GOV.UK, 2019; Citizens Advice, n.d.; Shelter, 2023).
- Watch suspicious words - we do pattern check on things such as bizarre deposit provisions or when the landlord wants you to repair everything. When something appears suspicious it brings out a note of possibly unfair and a Shelter reference (Shelter, 2023).
- Shelter England Topic library A list of frequently asked questions (repairs, deposits, Section 21/8, rent arrears) that is browsable. There are brief summaries, steps and links on every page.
- Local assistance- all you need to do is type in your postcode and it lists the closest councils and charities in a fast directory.
- Draft letters - simple letters (e.g. repair requests) can be auto-generated and copied and downloaded. Accounts (not required in MVP) - create an account or log in to websites favourite topics and draft letters so that they can be used later.

- Admin CMS - content editors have the ability to add or revise topics, notices and agencies.
- Every edit displays the time of the last update and the general notes of the editor.
- Analytics and feedback - page views and a fast rating / comment box will allow us to identify the pages that people see as confusing.

Non-functional:

- Usability and accessibility: we will cover the most fundamental (WCAG2.2AA) requirements: labels, contrast, focus states, key-board navigation. Source: (W3C, 2024; WAI, 2023.)
- Security & privacy - bare minimum data is stored; passwords are hashed, JWT to authenticate, rate limiting, access logs, there is a clear privacy statement.
- Accuracy and transparency - each source is displayed with the date; in case of the rule change (e.g. the new Renters Rights Act coming into effect), a banner appears with a link to the GOV.UK (GOV.UK, 2025).
- Performance - the UI is responsive and the API must complete common reads within less than 500ms, rudimentary caching of topic pages is on the table.
- Maintenance - we will develop using modular Flask blueprints, typed data models, linters and CI tests.

3.2. Risk analysis

- Risk description Likelihood Impact mitigation/ control responsibility The right law or policy (e.g., Renters' Rights Act) 2025 The law or advice can become outdated. (Medium High risk)
- Display last updated on all topics; banner leading to GOV.UK where new policy begins; review of content monthly. Content Editor Users confuse tool with formal legal advice Tenants may interpret our summary as legally binding. (High Medium risk)
- Have a not legal advice disclaimer, refer to Citizens Advice and Shelter to get more assistance.
- UX Lead Pattern rules may fail to pick up or emphasize clauses that have been pasted.(Medium risk)
- Use careful words ("may not be fair"), present sources, include a feedback button.
- AI / Rules Lead Privacy violation Data could include emails or saved texts of users.
- Use low hash passwords, use JWT authentication, use HTTPS, use minimise data, use UK GDPR.
- Security Lead Accessibility obstacles Disabled users can have problems.(Medium risk)
- Follow (WCAG 2.2 AA) keyboard nav and colour contrast test pre-release.
- Frontend Developer Unfinished or obsolete information destroys credibility. (Medium risk)
- Editor reviewing date-stamping each record. Content Editor Overload endpoints Bots would overload endpoints. (Minor Medium risk)
- Request rate cap; CAPTCHA on large forms; watch logs. Backend Engineer Failure in integration (e.g. map/agency lookup) 3 rd party postcode or map services may fail. (Medium risk)
- Fallback message, last-working data in the cache, uptime monitoring. Platform Lead Outage of hostings or databases impacts on urgent users. (Low High risk)

- Managed hosting (Azure App Service + MongoDB Atlas), health checks, backup. DevOps Lead Technical expertise or time limit Students can have issues with the tech stack.(Medium risk)
- Plan Learning in medium sprints; retain MVP lean; open-source libs. Project Owner.

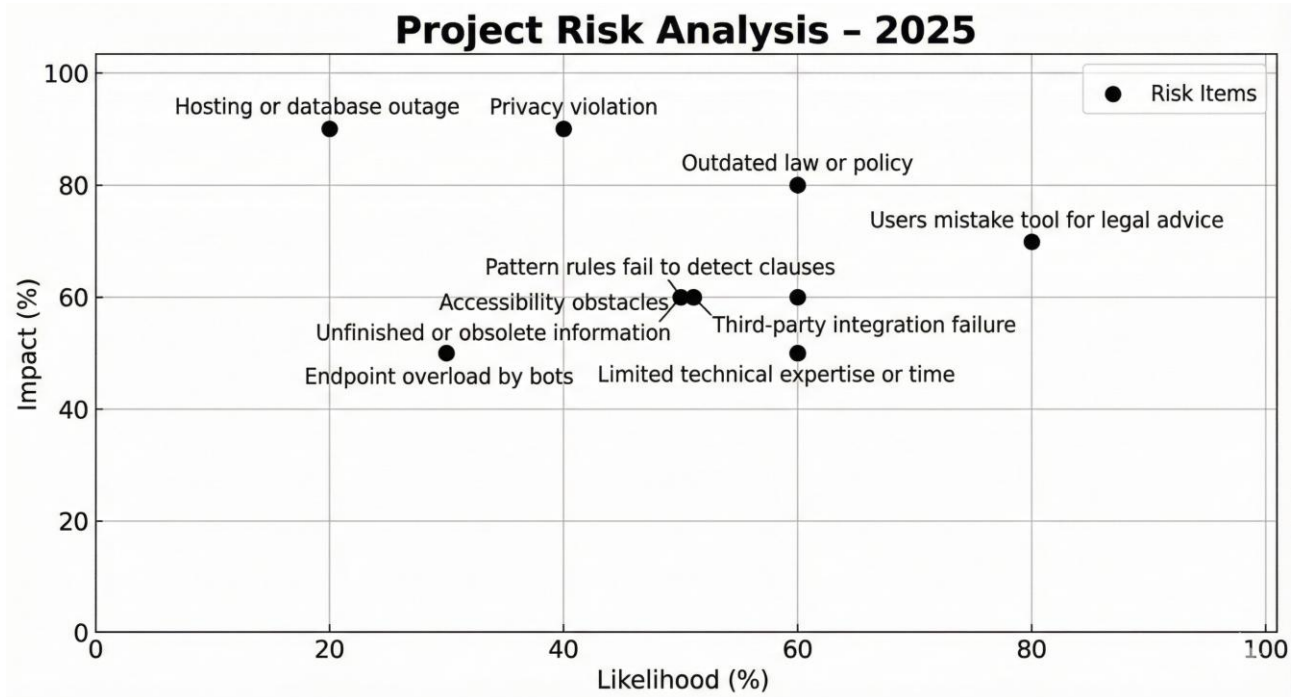
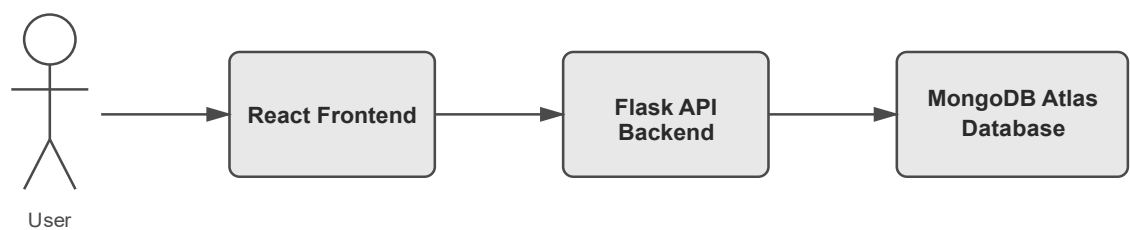


Fig: project risk analysis

This number provides a rough graphical overview of the key risks associated with our Home rights app project, indicating the score of each of them by probability and severity. It is simpler to convert the risks into percentages as this way, it becomes easier to see which parts may go wrong in our dev or annoy our users should we neglect them early. The stuff that has a high impact such as privacy issues and possible host or database outages would appear large on the chart, and it underlines the importance of good security and good infrastructure.

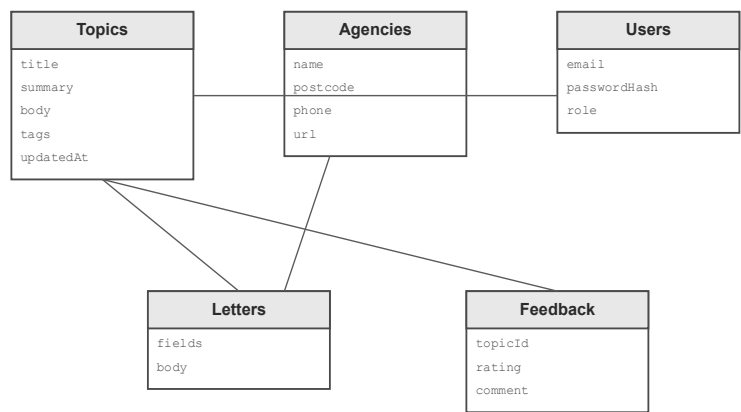
3.3. Design

System architecture of the Application



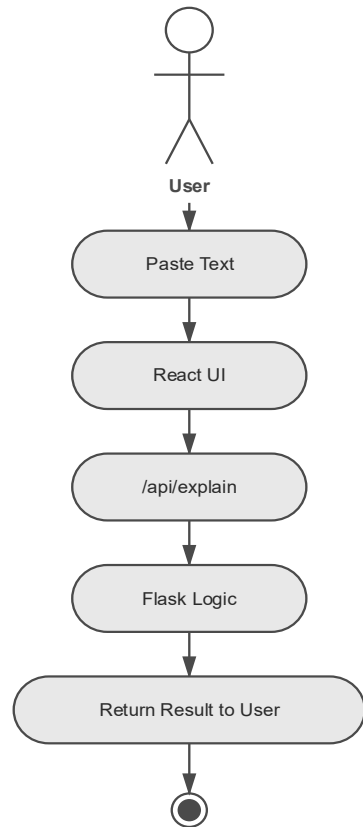
3.4.

Data model diagram of the Application



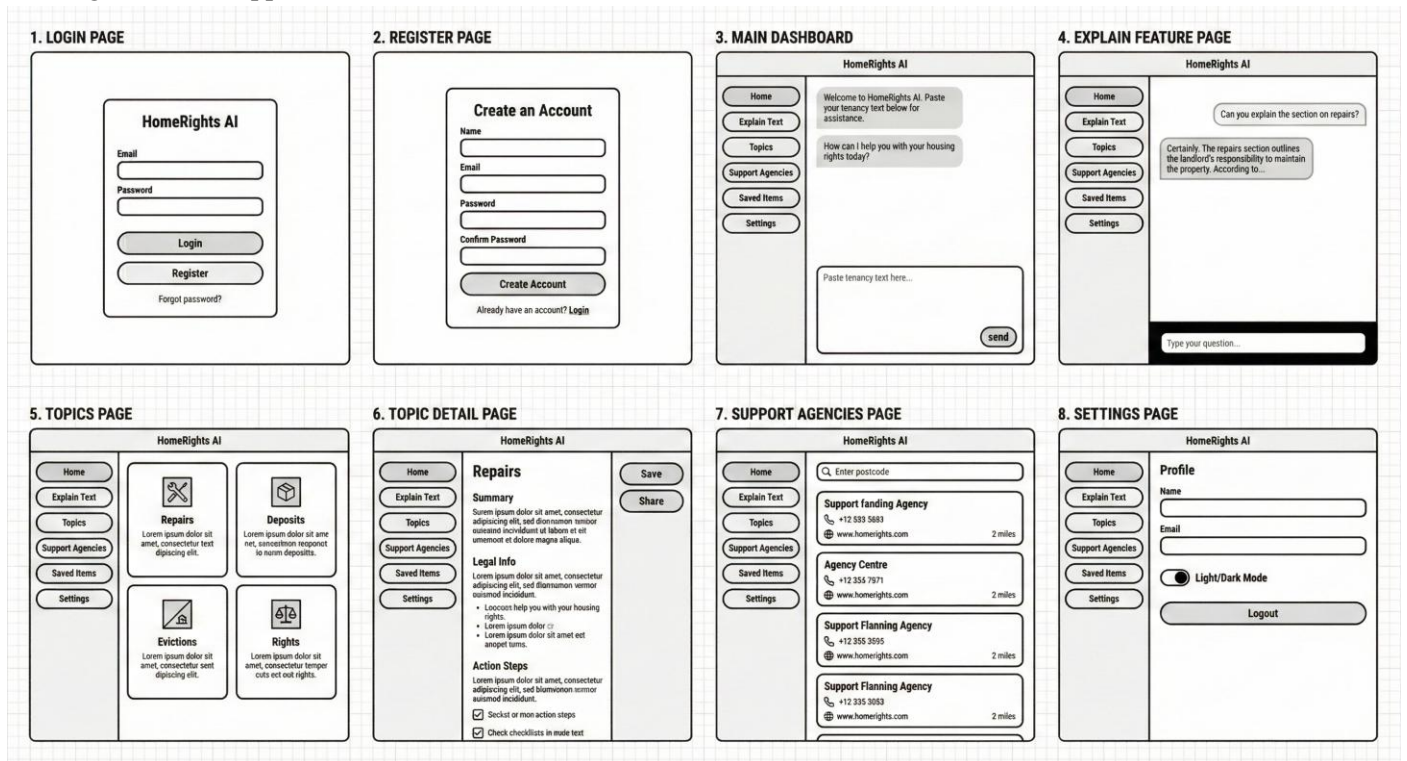
3.5.

UML Activity Diagram of the Application



3.6

UI Diagram of the Application



4. Planning

4.1. Methodology

- Sprint 0 (installation): repos; Flask + React template; MongoDB Atlas; CI pipeline.
- Sprint 1 (content + topics): Topics collection, Admin CMS basics, write first 5 topics (Repairs, Deposits, Rent arrears, Section 8, Section 21). Open to GOV.UK/Citizens Advice/Shelter and take notes of new dates (GOV.UK, 2019; Citizens Advice, n.d.; Shelter, 2023).
- Sprint 2 (Explain feature): paste: explanation; safe patterns; display uncertainty language; source links and dates.
- Sprint 3 (Local help): postcode search and list; basic agency editor.
- Sprint 4 (Letters): “fixes up draft;copy/download; add two more later.
- Sprint 5 (Accessibility and security): WCAG 2.2; JWT security; privacy notice (W3C, 2024; WAI, 2023). Sprint 6 (Testing and polish): usability testing, logging, analytics, bug fix. Release (MVP).

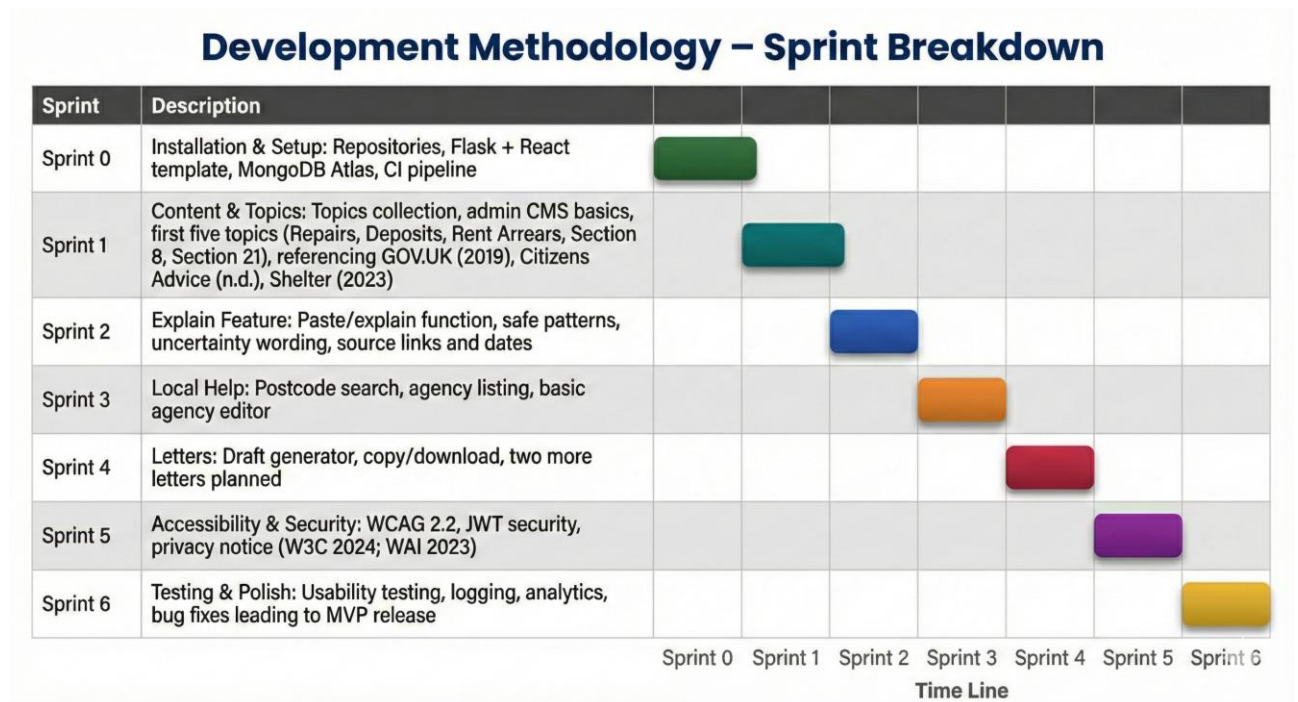


Fig: sprint Graph

This figure presents the schedule of our project as a study guide, which illustrates how we progressed through each sprint of the first set up to the last release of the MVP. The chart can assist us in visualizing the process of how we worked through an ordered, iterative workflow that is fairly similar to the Agile stuff we studied in class, by placing the work in a clear, black-and-white sequence. The initial sprints were dedicated to the establishment of the technical base, such as the Flask-React template, the connection to the database, and the development of the main content of the housing topics. The intermediate sprints are dedicated to the gradual development of the key functionality bit by bit, we have the explanation tool, the local help lookup, and the letter-generation system. Later sprints included accessibility, restricted security and testing before the prototype was finalized. All these visual representations allow one to understand how we ranked the tasks to do, how we dealt with dependencies and how each sprint propelled the project ahead. It also emphasizes the scholarly worth of demonstrating our growths in planning, pacing, and controlled features all through the entire developmental process.

4.2. Initial plans

The HomeRights AI project will take approximately 14 weeks, and has a simple agile-style time schedule. The work is divided into small sprints with the end of each of them consisting of a piece of working system and a brief review. The plan aims at the progressive development of a functional web application and at the same time gives time to research, testing, and reflection.

- Sprint 0 – Setup and Research (Weeks 1 -2), Build a Github repository and connect it to the Azure App Service to be deployed automatically. Install the background environment: Flask

(server), React (client), MongoDB Atlas (DBMS). Specify initial JSON topic, notice and support agencies data models. Create a mutual Figma layout of the user interface and select fonts and colours that comply with the WCAG 2.2 contrast. Seed content by using GOV.UK, Shelter and Citizens Advice as the initial sources of material. CI/CD workflow and test local hosting to make sure of fast deployment.

- Sprint 1 - Core Database and Topic Module (Weeks 3-4), Build Flask API endpoints to list and view housing-law topics. Enter the first five items: repairs, deposits, rent arrears, Section 8 notices and Section 21 notices. Add metadata fields including lastUpdated, sources and tags to each topic. Build React frontend pages of topic search and detail view. Test React/Flask connection with sample GET requests. Ensure that it loads fast and is readable on cell phones.
- Sprint 2: Describe Feature (Weeks 5-6), Add a Flask `/api/explain` endpoint. Create a simple textarea and an “Explain” button on the Paste Text page in React. Author Python logic (rule based) to identify key phrases (e.g., serve notice, deposit not protected). Create plain-English summaries and provide interconnected links to GOV.UK and Shelter. Colour code and tooltips on high risk words. Include error checking of blank input or excessive length of text.
- Sprint 3 - Local Support Directory (Weeks 7-8), Create the `/api/` support endpoint to find agencies by postcode. Add the names, addresses, phone and websites of all the agencies in MongoDB. Make use of a simple open postcode API to authenticate addresses. Create a React Find Help Near You page with cards, contact buttons. Provide admins with the option of updating or creating agency records on the CMS page. Enter various postcodes to make sure search is correct and quick.
- Sprint 4: Letter Generation (Weeks 9-10), Add a Flask endpoint `/api/letters/repair` to make repair request letters. Gather information of a basic form (name, address, problem, dates). HTML templates can be used and can be downloaded or copied to PDF. Give step-by-step instructions on how to send the letter to a landlord. Store the letters generated on the user account to be used later.
- Sprint 5 User Accounts and Authentication (Weeks 10-11), Use Flask JWT to implement register and login routes. Use short token expiry and Hash passwords with Passlib. Build React pages: Save topic and My Saved Items. Protect all the user-specific routes to make sure that only the logged-in users can gain access to them. Include the option to log out and automatic session logout. User flow Test user register on the site, and then log in, save, and log out.
- Sprint 6 - Accessibility and Testing (Weeks 12-13), Carry out an accessibility audit with Lighthouse and axe. Make sure that all pages are key-board only. Ask five or eight testers to carry out tasks (paste text, find help, create letter). Documented success rates and familiar areas of confusion. Address navigation and readability problems according to the feedback. Review security (HTTPS, JWT expiry, rate limits). Optimise API to maintain less than one second response times.
- Sprint 7 Evaluation and Final Documentation (Week 14). Re-test topic accuracy with GOV.UK and Shelter pages. Final content review with mentor and make updates. Make up evaluation summary in project report and presentation. Include graphs and screen shots of test outcomes. Prepare final user documentation and deployment documentation. Send code repository and backup database output.

Testing and Evaluation Postman and frontend forms will be used to perform functional testing during the development. Testing by the user will ensure that the tenants are able to past the text and comprehend the output without assistance. Manual keyboard walkthroughs and contrast checkers will be used in testing accessibility. Average response time and page loads will be measured by performance testing. The validation of the accuracy will be made against the official sources (GOV.UK, Citizens Advice, Shelter). Security checks will involve access/log out tests, expiry of tokens and rate-limit tests. The collection of feedback will be through a basic rating system on every topic.

References

1. Citizens Advice (no date) If you get a 'section 21' eviction notice (England). Available from: <https://www.citizensadvice.org.uk/housing/eviction/getting-evicted/renting-privately/if-you-get-a-section-21-notice/> [Accessed 12 November 2025].
2. GOV.UK (2025) Guide to the Renters' Rights Act. Available from: <https://www.gov.uk/government/publications/guide-to-the-renters-rights-act/guide-to-the-renters-rights-act> [Accessed 12 November 2025].
3. GOV.UK (2025) Historic Renters' Rights Act becomes law. Available from: <https://www.gov.uk/government/news/historic-renters-rights-act-becomes-law> [Accessed 13 November 2025].
4. GOV.UK (no date) Private renting for tenants: your rights and responsibilities. Available from: <https://www.gov.uk/private-renting> [Accessed 13 November 2025].
5. Shelter England (no date) Where tenancy rights come from. Available from: https://england.shelter.org.uk/professional_resources/legal/renting/introduction_to_security_of_tenure/where_tenancy_rights_come_from [Accessed 13 November 2025].
6. Shelter England (no date) Section 21: getting the dates right. Available from: https://england.shelter.org.uk/professional_resources/news_and_updates/section_21_getting_the_dates_right [Accessed 13 November 2025].
7. Legislation.gov.uk (2025) Renters' Rights Act 2025 (c. 26). Available from: <https://www.legislation.gov.uk/ukpga/2025/26/contents/enacted> [Accessed 13 November 2025].

Appendices

Appendix A – System Diagrams

System Architecture Diagram

Data Model Diagram(UML Diagram)

Flow Diagram

Appendix B – API Endpoints (Evidence List)

/api/topics

/api/topics/{id}

/api/explain

/api/support

/api/letters/repair

/api/auth/register

/api/auth/login

Appendix C – Database Fields (Evidence List)

Topics: title, summary, body, tags, sources, updatedAt

Agencies: name, postcode, phone, url

Users: email, passwordHash, role, savedItems

Feedback: topicId, rating, comment, createdAt

Appendix D – Testing Evidence

Postman screenshots of successful API calls

Lighthouse accessibility score screenshot

Console log showing backend connection

User testing checklist (completed)

Appendix E – Sample Output

Sample repair request letter (generated)

Example Explain Feature output (redacted)

Appendix F – Source Links Used (Evidence List)

GOV.UK: <https://www.gov.uk/private-renting/your-rights-and-responsibilities>

Citizens Advice: <https://www.citizensadvice.org.uk/housing/>

Shelter England: <https://england.shelter.org.uk/>

Legislation.gov.uk: <https://www.legislation.gov.uk/ukpga/2025/26/contents/enacted>
WCAG 2.2: <https://www.w3.org/TR/WCAG22/>